Example Code: 60-Second Sweep

Have you ever wanted a clock that moves backwards? For our example source code found in Listing 9-6, we are going to create a pseudo time-keeping device that is eternally trapped in a 60-second loop. The stepper will move in a full 360º arc in about a minute. At the end of its arc, it will switch direction and spin the other way. Because our stepper moves in 1.8º steps, it's not entirely possible to have the most exacting of timepieces here, but it's close enough. Because of this, each second the stepper should move 3.33 steps, which it can't do. So we fudge the delay a little bit with a delay between steps of 909 milliseconds instead of 1 second. Because of how the two different circuits shown in the figures are designed, the single sketch provided will control either type of motor.

Listing 9-6. *60-Second Sweep Source Code*

```
#include <Stepper.h>

const int steps = 200;
Stepper stepper(steps, 8, 9, 10, 11);

int stepDirection = 3;
int counter = 0;

void setup() {
  stepper.setSpeed(30);
}

void loop() {
  stepper.step(stepDirection);
  delay(909);
  counter+=3;
  if (counter > steps) {
    counter = 0;
    if (stepDirection == 3) stepDirection = -3;
    else stepDirection = 3;
  }
}
```

With our simple code in place rotating our stepper motor back and forth, let's explore the Stepper functions in greater depth.

Stepper

To create a new instance of the Stepper library, we use one of the following two syntaxes:

```
Stepper name(steps, pin 1, pin 2)
Stepper name(steps, pin1, pin 2, pin 3, pin4)
```

The syntax we use depends on whether we are using two pins or four pins to control the stepper drivers. To make our wiring a little easier and the code work on both drivers, we chose to use four pins—easier to wire at the cost of a pair of digital pins. In our example code, we created the following `Stepper` instance: